

RELATÓRIO

Aluno: Isabel E. Valenzuela Illanes

Turma: 1G

Curso: Técnico Integrado em Informática para Internet

Professor: Hewerton Enes de Oliveira

1. IDEIA PRINCIPAL

A ideia principal do projeto foi desenvolver um site sobre as músicas da cantora japonesa ADO, utilizando o projeto como uma oportunidade para testar e aprofundar o conhecimento em HTML e CSS. O foco técnico foi trabalhar principalmente a organização estética da página, a utilização de efeitos visuais, o chamado fading (efeito de transição de opacidade) e a criação de um carrossel de imagens.

Além de apresentar informações sobre as músicas, o objetivo foi fazer com que o site tivesse uma aparência mais dinâmica e interativa. Para isso, foram utilizados recursos do CSS para que o usuário pudesse selecionar diferentes capas de músicas e, a partir dessa seleção, visualizar informações correspondentes e alterar o fundo da página.

Uma das principais escolhas do projeto foi realizar o carrossel utilizando apenas HTML e CSS, sem depender de JavaScript. Dessa forma, foi possível explorar recursos como `:checked`, o combinador `~`, transições, Flexbox e CSS Scroll Snap.

2. CARROSSEL

Uma das partes principais do projeto foi a criação do carrossel de capas das músicas. O código utilizado foi:

```
.carrossel {
  display: flex;
  gap: 16px;
  overflow-x: auto;
  scroll-snap-type: x mandatory;
  padding: 4px 4px;
}

.carrossel label {
  flex: 0 0 200px;
  scroll-snap-align: center;
  cursor: pointer;
}
```

```
border: 2px solid transparent;
opacity: 0.6;
transition: opacity 0.3s, transform 0.3s, border-color 0.3s;
}

.carousel img {
display: block;
width: 100%;
aspect-ratio: 1 / 1;
object-fit: cover;
}

.carousel label:hover { opacity: 1; }
```

O `display: flex` foi utilizado para organizar as capas das músicas em uma única linha. O comando `gap: 16px` cria uma separação de 16 pixels entre cada uma das capas, evitando que elas fiquem muito próximas umas das outras.

O `overflow-x: auto` permite que, caso todas as capas não caibam na largura disponível da página, seja possível utilizar a rolagem horizontal para visualizar as demais. Esse recurso é uma das características que fazem com que o conjunto de imagens funcione como um carrossel.

Já o `scroll-snap-type: x mandatory` faz com que a rolagem horizontal tenha pontos de encaixe. Dessa forma, quando o usuário para de arrastar o carrossel, a imagem tende a ficar alinhada corretamente, em vez de permanecer posicionada no meio de duas capas.

O `scroll-snap-align: center` determina que o ponto de encaixe de cada capa seja o seu centro. Esses dois comandos funcionam em conjunto para deixar a movimentação do carrossel mais organizada.

O `border: 2px solid transparent` cria um espaço reservado para a borda, porém inicialmente ela fica transparente. Isso é importante porque, quando a capa selecionada recebe uma borda branca, ela não altera seu tamanho nem desloca os outros elementos.

A propriedade `opacity: 0.6` faz com que as capas apareçam inicialmente com uma aparência mais apagada. Quando o usuário passa o mouse sobre uma delas, o comando `:hover` aumenta sua opacidade para 1, fazendo com que a imagem fique mais destacada.

Por fim, a propriedade `transition` cria uma animação suave para as mudanças de opacidade, transformação e cor da borda. O tempo de `0.3s` evita que as alterações aconteçam de maneira brusca.

3. SELEÇÃO DA MÚSICA

No projeto inicial, o carrossel havia sido pensado com o uso de JavaScript. Entretanto, durante o desenvolvimento, foi possível substituir essa lógica por recursos do próprio CSS.

Para isso, foram utilizados elementos `input` do tipo `radio` juntamente com a pseudoclasse `:checked`. O código utilizado para destacar a música selecionada foi:

```
#s1:checked ~ main label[for="s1"],
#s2:checked ~ main label[for="s2"],
#s3:checked ~ main label[for="s3"],
#s4:checked ~ main label[for="s4"] {
  opacity: 1;
  transform: scale(1.05);
  border-color: #fff;
}
```

Esse código permite identificar qual música foi selecionada sem a necessidade de JavaScript. Quando um dos `input` é marcado, o CSS procura o `label` correspondente e aplica os efeitos visuais.

A imagem selecionada fica um pouco maior através do `transform: scale(1.05)` e recebe uma borda branca por meio de `border-color: #fff`. Ao mesmo tempo, sua opacidade passa para 1, fazendo com que ela fique mais evidente em relação às outras capas.

Além disso, a mesma lógica foi utilizada para fazer com que o quadro de informações correspondente a cada música apareça de acordo com a seleção. Dessa maneira, quando o usuário escolhe outra capa, as informações também são atualizadas, criando uma sensação de interação sem precisar utilizar JavaScript.

4. FUNDOS

Outra parte importante do projeto foi a criação dos fundos correspondentes às capas das músicas. Para isso, foram utilizados elementos posicionados atrás do conteúdo

```
.fundos div {
  position: fixed;
  inset: 0;
  z-index: -1;
  background-size: cover;
  background-position: center top;
  opacity: 0;
```

```
transition: opacity 0.8s ease;

-webkit-mask-image: linear-gradient(to bottom, #000 30%, transparent
100%);
mask-image: linear-gradient(to bottom, #000 30%, transparent 100%);
}
```

O **position: fixed** mantém o fundo fixo na tela, enquanto o **inset: 0** faz com que ele ocupe toda a área disponível. O **z-index: -1** coloca o fundo atrás dos outros elementos da página.

O **background-size: cover** permite que a imagem cubra toda a área disponível sem deixar espaços vazios. Já o **background-position: center top** controla a posição da imagem para que a parte principal fique melhor enquadrada.

Para criar o efeito de fading, foi utilizada a propriedade **opacity** juntamente com **transition**. Dessa forma, quando o fundo muda, ele não desaparece ou aparece de maneira repentina. A alteração acontece gradualmente durante aproximadamente 0.8 segundos.

Também foi utilizada uma máscara com **linear-gradient**, que faz com que a imagem fique gradualmente transparente em determinada parte da tela. Isso ajuda a misturar o fundo com o restante do conteúdo e melhora a leitura das informações.

Para fazer com que o fundo correspondente apareça apenas quando a música é selecionada, foi utilizado:

```
.f2 { background-image: url(img/cover2.jpeg); }
.f3 { background-image: url(img/cover3.jpeg); }
.f4 { background-image: url(img/cover4.jpeg); }

#s2:checked ~ .fundos .f2,
#s3:checked ~ .fundos .f3,
#s4:checked ~ .fundos .f4 {
  opacity: 0.9;
}
```

Esse código faz com que a mesma imagem utilizada na capa da música também apareça no fundo da página, porém com uma opacidade menor e com o efeito de fading. O primeiro item foi retirado dessa lógica porque, durante os testes, considerei que o resultado visual ficava mais agradável mantendo apenas o fading padrão do **body**. Dessa maneira, a primeira imagem não teria uma repetição tão forte da capa no fundo.

5. PROBLEMAS ENCONTRADOS DURANTE O PROCESSO

Durante o desenvolvimento do projeto, foram encontrados diversos problemas relacionados principalmente ao posicionamento das imagens, à organização dos elementos e à ligação entre o HTML e o CSS.³⁻⁴

Um dos primeiros problemas aconteceu com o posicionamento das imagens. Inicialmente, algumas capas não estavam ficando corretamente alinhadas e os espaços entre elas não apresentavam o resultado esperado. Foram realizados alguns testes alterando manualmente valores de pixels, como largura, espaçamento e tamanho das imagens. Porém, essas alterações acabaram deixando o resultado visual ainda pior, pois o problema não estava exatamente nas dimensões dos elementos.

Depois de verificar o código com mais atenção, foi descoberto que parte do problema estava relacionada ao nome e à extensão dos arquivos das imagens. Algumas imagens haviam sido salvas com a extensão `.jpeg`, enquanto no código CSS elas estavam sendo chamadas utilizando `.jpg`. Apesar de serem formatos muito semelhantes, o nome do arquivo precisa corresponder exatamente ao arquivo existente na pasta. Como havia essa diferença, o navegador não conseguia encontrar corretamente algumas imagens, fazendo com que o fundo não fosse carregado.¹

Esse problema mostrou a importância de conferir não apenas o código CSS, mas também a organização dos arquivos do projeto. Uma pequena diferença no nome de um arquivo pode fazer com que uma imagem ou outro recurso deixe de funcionar completamente.

Outro problema aconteceu com a última imagem do carrossel, que aparecia em uma posição incorreta, ficando abaixo da penúltima. Nesse caso, o problema não estava relacionado ao Flexbox ou ao tamanho da imagem, mas sim a um erro de digitação no código HTML. Algumas aspas estavam incorretas e não permitiam que uma tag `<label>` fosse fechada corretamente. Como consequência, o navegador interpretava parte do código de uma maneira diferente da esperada, alterando a estrutura da página.²

A correção das aspas e do fechamento das tags fez com que o HTML voltasse a ser interpretado corretamente e resolveu o problema do posicionamento.

Também foram encontrados problemas na estrutura geral dos efeitos gráficos, principalmente durante a tentativa de fazer o fundo mudar de acordo com a música selecionada. Foi necessário testar diferentes valores de opacidade, tamanhos, posições e tempos de transição até encontrar uma combinação que não prejudicasse a leitura do conteúdo.

Outro desafio foi fazer com que todos esses efeitos funcionassem juntos utilizando apenas CSS. Como o projeto precisava responder à seleção de cada música, foi necessário compreender melhor como funcionam a pseudoclasse `:checked` e o

combinador de irmãos gerais ~. Esses recursos permitiram controlar diferentes elementos da página dependendo do input selecionado.

Durante o desenvolvimento, também foi utilizado o recurso de Inteligência Artificial, especificamente o Claude, como ferramenta auxiliar para depuração do código. A IA foi utilizada principalmente para identificar erros, sugerir correções e ajudar a melhorar partes do código que não estavam funcionando corretamente. No entanto, as sugestões precisaram ser testadas e adaptadas ao projeto, pois nem todas as alterações apresentadas produziam o resultado visual esperado.

Esse processo foi importante porque permitiu compreender melhor os erros encontrados e perceber que, muitas vezes, o problema não está necessariamente no recurso utilizado, mas em pequenos detalhes da estrutura do HTML ou CSS.

6. RECURSOS E CONCEITOS PESQUISADOS

O que fazer	Como se chama	O que pesquisar na MDN
Portadas em fila	Flexbox	<code>display: flex, flex</code>
Fazer o scroll parar em cada portada	CSS Scroll Snap	<code>scroll-snap-type, scroll-snap-align</code>
Detectar a música escolhida	Pseudoclasse	<code>:checked</code>
Afetar um elemento que vem depois no HTML	Combinador irmãos gerais	de ~
Animar os cambios	Transições	<code>transition</code>
Fundo fixo atrás do conteúdo	Posicionamento	<code>position: fixed, z-index</code>
Fading do fundo	Máscaras CSS degradados	e <code>mask-image, linear-gradient</code>
Imagem quadrada sem deformação	Proporção e ajuste de objeto	<code>aspect-ratio, object-fit</code>

7. VERSÕES DO PROJETO

Durante o desenvolvimento, foram produzidas diferentes versões do site. A versão semi-final foi utilizada para testar os principais recursos e identificar problemas de organização, posicionamento e funcionamento.

Depois dos testes e das correções, foi desenvolvida a versão final, com os elementos organizados de maneira mais adequada e com os efeitos de carrossel, seleção, fading e mudança de fundo funcionando em conjunto.

A comparação entre as versões também ajudou a perceber a evolução do projeto, principalmente na parte visual e na organização do código.

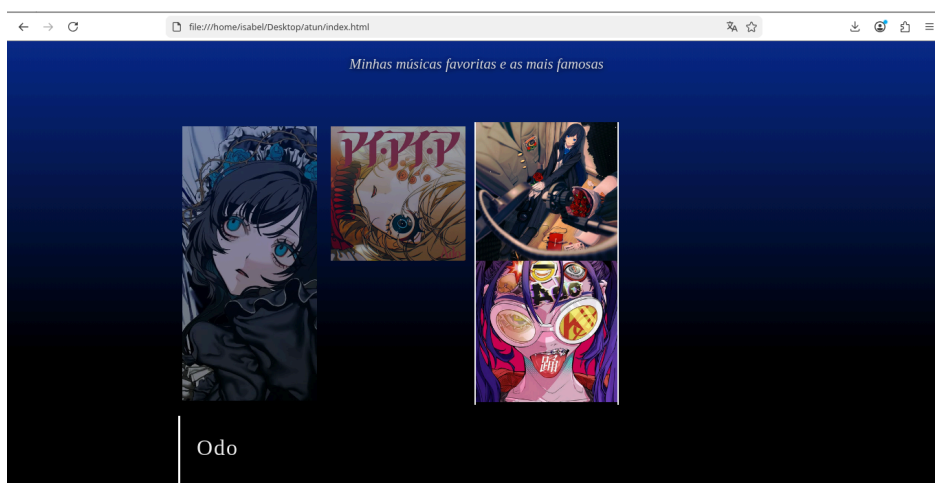
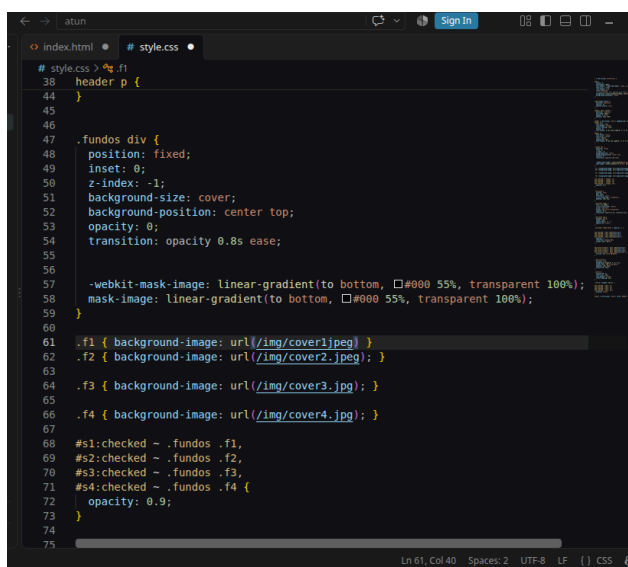
8. CONCLUSÃO

O desenvolvimento deste projeto permitiu colocar em prática diversos conceitos de HTML e CSS, principalmente relacionados à organização visual, posicionamento, transições e interação com o usuário.

Um dos principais aprendizados foi perceber que é possível criar efeitos relativamente complexos sem necessariamente utilizar JavaScript. O uso de `:checked`, `~`, Flexbox, CSS Scroll Snap, `transition`, `mask-image` e `linear-gradient` possibilitou criar um site interativo e visualmente mais elaborado utilizando principalmente recursos do CSS.

Além disso, os problemas encontrados durante o desenvolvimento foram importantes para o aprendizado. Erros simples, como extensões incorretas de imagens ou aspas mal colocadas no HTML, tiveram efeitos grandes no funcionamento da página. A necessidade de testar diferentes soluções também ajudou a compreender melhor a relação entre HTML e CSS

9.Imagens



Diagnosing why some carousel background images fail to load.

Los tres casos tienen causas distintas. Dos son los errores que dejé a propósito en el código, y el tercero casi seguro es el nombre del archivo.

Tercera imagen (ERRO 3): en `style.css` busca esta línea:

```
css
.f3 { background-image: url(/img/cover3.jpg); }
```

El `/` inicial hace que el navegador busque en la raíz de tu computadora (`/img/...`) y no en tu carpeta. Quitálo:

```
css
.f3 { background-image: url(img/cover3.jpg); }
```

Cuarta imagen (ERRO 4): el fondo no cambia porque el clic nunca selecciona esa canción. En `index.html` busca:

```
html
<label for="s04">
```

y cámbialo a:

```
html
<label for="s4">
```

Con esto la 4ª canción también muestra su información.

Primera imagen: la 1ª y la 2ª usan la misma lógica, y la 2ª sí funciona. Por eso el problema debe ser que el nombre del archivo no coincide con el del CSS. Pasa, por ejemplo, si tu archivo se llama `cover1.png` o `Cover1.jpg`, o si en el HTML pusiste otro nombre y no lo cambiaste en el CSS.

Por lo que se ve en la captura, el CSS del carrusel y de los fondos no se está aplicando. Casi seguro es un problema de nombres de clase.

Cómo lo sé:

- Las portadas aparecen apiladas en vez de en una fila con scroll horizontal, y la 4ª cae debajo de la 3ª. Eso pasa cuando `display: flex` no llega al contenedor.
- El borde blanco de la primera portada baja como una columna larga. Esa regla sí funciona porque no depende de ninguna clase (`#s1:checked ~ main label[for="s1"]`), pero el `<label>` quedó como elemento en línea sin el resto del estilo.
- El fondo no muestra la portada de la canción seleccionada, aunque la 1 está activa.

Qué pasó: en mi primera versión las clases estaban en español (`carrusel`, `fundos`) y en la segunda las pasé a portugués (`carrossel`, `fundos`). Si mezclaste partes de las dos versiones, el HTML dice una cosa y el CSS otra, y el CSS no encuentra nada a lo que aplicarse.

Cómo resolverlo:

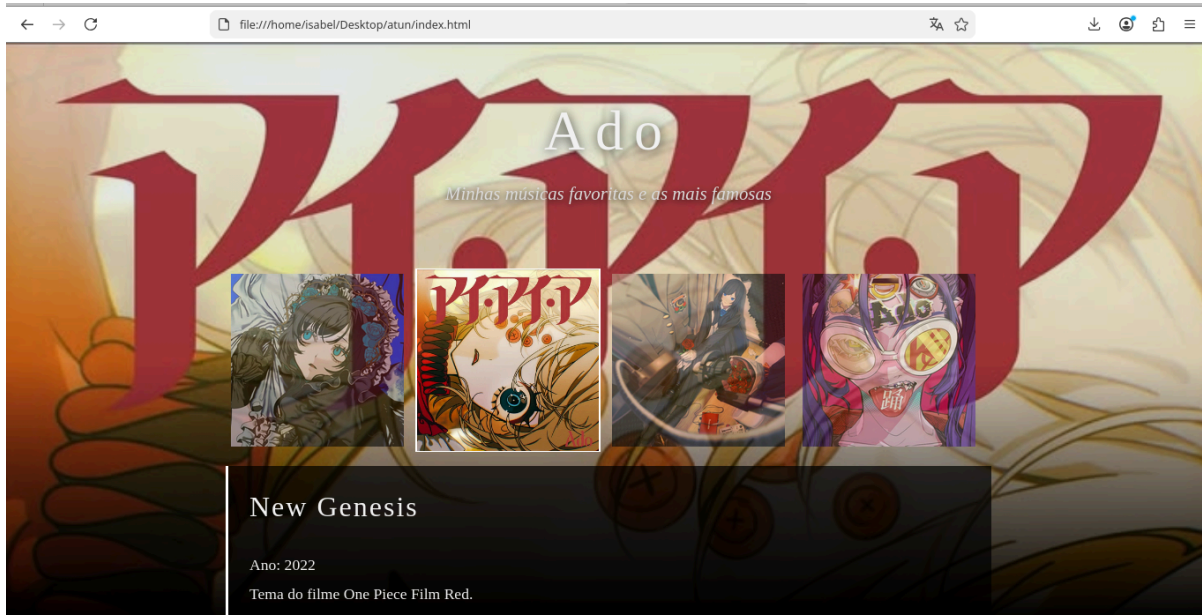
- Abre `index.html` en VS Code y busca estas clases. Deben quedar exactamente así:

```
html
<div class="fundos">
<section class="carrossel">
<section class="info">
```

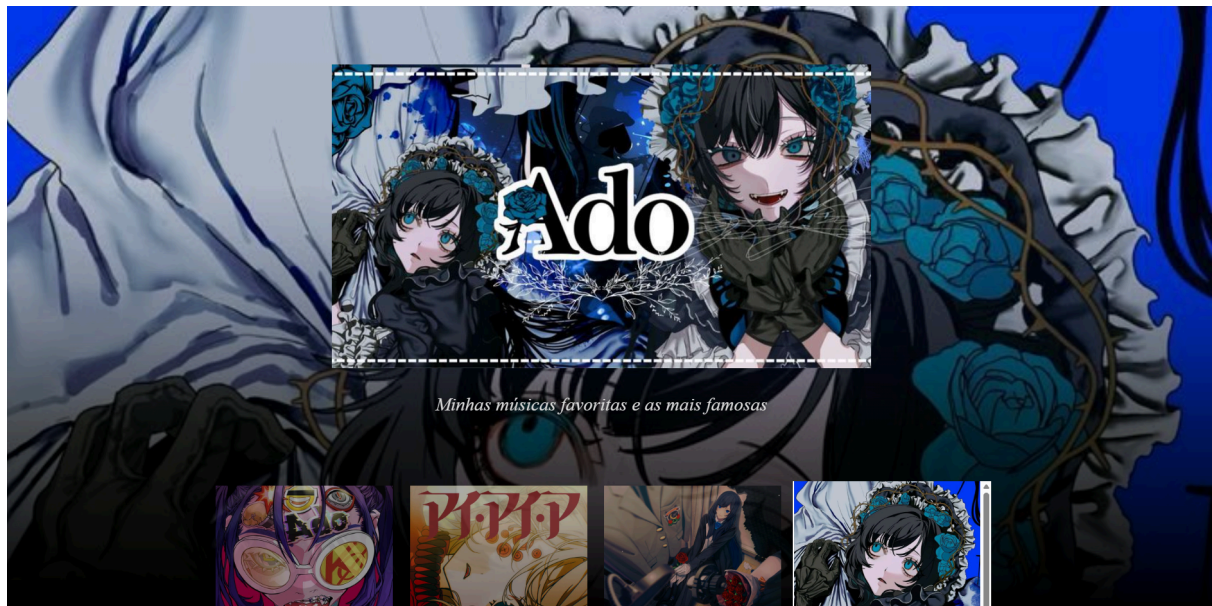
Si ves `fundos` o `carrusel`, cámbialas por `fundos` y `carrossel`.

- Abre `style.css` y confirma que usa `.fundos` y `.carrossel`, con las dos "s" y la "o", y no `.carrusel` ni `.fundos`.
- Si prefieres no revisar línea por línea, vuelve a descargar mis dos archivos de la respuesta anterior. Reemplaza `index.html` y `style.css` en tu carpeta `atun`, conservando la carpeta `img/`. Entre ellos las clases ya coinciden.
- Recarga con `Ctrl + Shift + R`, para que el navegador no use el CSS guardado en caché.

versao semi-final



Site final



10. REFERÊNCIAS

<https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Overflow/Carousels>

<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/:checked>

<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/z-index>

oopen.io/mwankhede/pen/VbyyRN